

# ХБК АКСАЙ

Проект AI Sales Director

## ПЛАН РАБОТЫ — ЭТАП 2

Внешний интерфейс: Дашборд аналитика и AI-чат

Версия 1.0 · Апрель 2026

Подготовлено: Benevsky Team

РЕЗЮМЕ ДЛЯ РУКОВОДСТВА

Настоящий документ фиксирует план реализации Этапа 2 проекта AI Sales Director для ХБК Аксай. Этап 2 предполагает создание двух пользовательских интерфейсов поверх уже функционирующего AI-ядра (ETL-пайплайн 1С → PostgreSQL → LLM+RAG), без изменения существующей инфраструктуры данных.

**Ключевые параметры Этапа 2**

**Срок реализации:** 6 недель (3 спринта по 2 недели)

**Два интерфейса:** Дашборд аналитика (руководство, аналитики) + AI-чат с пресетами (отдел продаж)

**Frontend-стек:** React + TypeScript + shadcn/ui (готовый шаблон) + Recharts/Tremor

**Backend-слой:** FastAPI (API Gateway) + Celery + Redis — паттерны заимствованы из архитектуры Allosta v2.0

**CoreGit:** Версионированная файловая система для AI-агентов — хранение и аудит AI-выводов, пресетов, конфигураций

**База данных:** Существующая PostgreSQL — только чтение; схема не изменяется

**Пресеты промптов:** 28 готовых шаблонов запросов, разбитых по 5 категориям работы с клиентами

Принципиальное решение Этапа 2 — не разрабатывать UI с нуля. Использование готового shadcn/ui-шаблона (dashboard + chat) сокращает трудозатраты на вёрстку на 3–4 недели. Единственная точка сложности — интеграция SSE-стриминга ответов LLM и корректная маршрутизация RAG-контекста.

**КОНТЕКСТ: СУЩЕСТВУЮЩАЯ АРХИТЕКТУРА AI SALES DIRECTOR**

2.1 Что уже работает (Этап 1)

Этап 1 сформировал полноценное AI-ядро. Этап 2 строится поверх него без вмешательства в данные или модели.

| Компонент                                   | Статус   | Роль в Этапе 2                         |
|---------------------------------------------|----------|----------------------------------------|
| ETL-пайплайн (1C → PostgreSQL)              | Работает | Источник данных — только чтение        |
| PostgreSQL (схема клиентов, RFM, заказы)    | Работает | Единый источник правды                 |
| LLM-сервер (Llama3/Mistral, локальный хост) | Работает | Генерация ответов в чате и AI Insights |
| RAG-слой (векторная БД + retrieval)         | Работает | Обогащение запросов данными клиента    |
| Churn Prediction Engine (0–100%)            | Работает | Данные для дашборда и списка действий  |
| RFM-сегментация (Core/A/B/C)                | Работает | RFM-матрица и фильтры дашборда         |
| WhatsApp / Telegram Bot API                 | Работает | Генерация сообщений через AI-чат       |

Этап 2 не изменяет ни одну из перечисленных систем. Новый API Gateway работает с существующей PostgreSQL в режиме read-only, вызывая LLM через существующий HTTP-интерфейс.

2.2 Что добавляет Этап 2

Этап 2 создаёт три новых компонента: API Gateway (тонкий адаптер между фронтом и AI-ядром), Frontend (два React-приложения на shadcn/ui), и CoreGit-интеграцию (версионирование AI-выводов).

| Новый компонент     | Технология                   | Назначение                                          |
|---------------------|------------------------------|-----------------------------------------------------|
| API Gateway         | FastAPI + JWT                | Единая точка входа, аутентификация, маршрутизация   |
| Async Worker        | Celery + Redis               | Фоновая генерация AI Insights (крон), очередь задач |
| Дашборд аналитика   | React + shadcn/ui + Recharts | Аналитика, тренды, список действий                  |
| AI-чат с пресетами  | React + shadcn/ui + SSE      | Ответы LLM, 28 пресетов, история чатов              |
| CoreGit-репозиторий | @coregit/sdk (TypeScript)    | Версионированное хранилище AI-выводов               |

API Gateway использует паттерн из архитектуры Allosta v2.0: один FastAPI-сервис заменяет несколько CRM-мостов и устраняет vendor lock-in. Принцип тот же — один PostgreSQL как источник правды, без внешних SaaS-зависимостей.

# COREGIT: ВЕРСИОНИРОВАННАЯ ФАЙЛОВАЯ СИСТЕМА ДЛЯ AI-АГЕНТОВ

## 3.1 Что такое CoreGit

CoreGit — это не backend-фреймворк и не CRM-заменитель. CoreGit предоставляет версионированную файловую систему для AI-агентов через единый REST API, построенный поверх git. Ключевая характеристика: атомарный коммит до 100 файлов за 1 API-вызов (19.8 сек против 72.1 сек у GitHub), пропускная способность 15 000 коммитов/час, стандартный git-протокол для клонирования и pull.

**Ключевые возможности CoreGit (по документации)**  
Atomic commits — коммит множества файлов одним REST-вызовом (POST /v1/repos/{slug}/commits)  
Filesystem mount — монтирование репозитория как ФС через TypeScript SDK (@coregit/sdk), запись файлов, выполнение bash-команд внутри снапшота  
Snapshots & restore — именованные точки восстановления; откат до любого состояния одним вызовом  
Branch, diff, merge — полный git-воркфлоу через API; сравнение изменений, fast-forward merge  
Semantic search — встроенный поиск по смыслу (embeddings + Tree-sitter code graph); не нужен отдельный Elasticsearch  
Standard git protocol — git clone / push / pull работает с любым стандартным git-клиентом  
\$0 при простое — pay-per-use; нет сервера для поддержки

## 3.2 Роль CoreGit в архитектуре Этапа 2

В проекте AI Sales Director CoreGit выполняет строго определённую роль: версионированное хранилище артефактов AI-агентов. Это не замена PostgreSQL и не API-сервер — это git-репозиторий для всего, что генерирует AI.

| Сценарий использования            | Что коммитится в CoreGit                            | Зачем                                               |
|-----------------------------------|-----------------------------------------------------|-----------------------------------------------------|
| AI Insights (ежедневные)          | JSON с тезисами + markdown-отчёт по каждому запуску | Полная история всех AI-рекомендаций с датой         |
| Список действий ("кому написать") | action_list_YYYY-MM-DD.json                         | Аудит: какой совет давал AI в конкретный день       |
| Пресеты промтов                   | presets.json с 28 шаблонами                         | Версионирование; откат при нежелательных изменениях |
| Пороговые конфигурации            | thresholds.json (churn_threshold, rfm_weights)      | История изменений параметров модели                 |
| Экспортированные отчёты           | PDF/CSV-отчёты по периодам                          | Долгосрочный архив без нагрузки на PostgreSQL       |
| Снапшоты перед изменением модели  | Снапшот репозитория через snapshot API              | Гарантированный откат при деградации качества       |

CoreGit не обрабатывает HTTP-запросы от фронтенда. Взаимодействие происходит только внутри backend-воркера (Celery-задача вызывает @coregit/sdk при фиксации AI-вывода). С точки зрения пользователя CoreGit невидим.

## 3.3 Паттерн интеграции

Интеграция реализуется в Celery-воркере (Node.js subprocess или прямой TypeScript-сервис).  
Схема: Celery-задача генерирует AI-вывод → формирует JSON/markdown-файлы → вызывает CoreGit SDK для атомарного коммита → возвращает sha коммита в PostgreSQL (поле last\_coregit\_sha в таблице insights).

Поиск по истории AI-рекомендаций — через CoreGit Semantic Search API (встроенные embeddings). Запрос: "какие клиенты из сегмента А получали рекомендацию реактивации в Q1 2026" — не требует кастомного поискового движка.

АРХИТЕКТУРНЫЕ ПАТТЕРНЫ ИЗ ALLOSTA V2.0

4.1 Ключевой вывод Allosta

Архитектура Allosta Partners v2.0 (апрель 2026) решала аналогичную задачу: заменить двойной CRM-стек (Bitrix24 + amoCRM с двунаправленной синхронизацией) единым источником правды на PostgreSQL + React. Итог: Time-to-Market -30%, устранение vendor lock-in на две платформы, снятие rate-limit ограничений для AI-агента.

Для AI Sales Director Этапа 2 ситуация симметрична: не создавать лишних сервисов, один PostgreSQL — источник правды, весь UI — кастомные React-компоненты без внешних SaaS-зависимостей.

| Паттерн Allosta                               | Применение в Этапе 2                                   | Что это даёт                                             |
|-----------------------------------------------|--------------------------------------------------------|----------------------------------------------------------|
| Единый PostgreSQL вместо множества источников | Только одна БД — существующая; нет дополнительных БД   | Нет синхронизации, нет расхождения данных                |
| FastAPI как лёгкий API Gateway                | API Gateway на FastAPI — единая точка входа            | Async-ready, автодокументация (Swagger), низкий overhead |
| Celery + Redis для фоновых задач              | Ежедневная генерация AI Insights, очередь LLM-запросов | Фронт не ждёт тяжёлых вычислений; retry при сбоях        |
| Scoring Engine как отдельный сервис           | Существующий Churn Engine вызывается через HTTP        | Не трогаем рабочий код; слабая связанность               |
| React без vendor lock-in                      | shadcn/ui (Radix UI + Tailwind) — нет платного SaaS    | Полный контроль кода, нет rate limits                    |
| Роли через JWT                                | analyst / sales / admin                                | Один токен, один gateway, два профиля доступа            |

Паттерны не переносятся механически — адаптируются под масштаб. В Allosta была корпоративная платформа с investor pipeline. Для AI Sales Director достаточно двух ролей и трёх сервисов. Главное правило: не добавлять сложности раньше, чем она понадобится.

## ИНТЕРФЕЙС 1: ДАШБОРД АНАЛИТИКА

### 5.1 Шаблон и базовая структура

За основу берётся официальный shadcn/ui Dashboard Template (ui.shadcn.com/examples/dashboard). Шаблон содержит готовые компоненты: sidebar navigation, top bar, card grid, data tables, responsive layout. Адаптация под данные ХБК Аксай занимает 3–4 дня вместо 2–3 недель с нуля.

Структура навигации (боковое меню):

- Главная (Overview) — KPI-карточки, график выручки, AI Insights, список действий
- Клиенты — поиск, фильтрация, карточка клиента, история взаимодействий
- Аналитика — тренды, сравнение периодов, RFM-матрица, сезонность
- Экспорт — PDF-отчёты, CSV-выгрузки
- Настройки — пороги уведомлений, веса RFM, фильтры по умолчанию (только admin)

### 5.2 Компоненты главной страницы

| Компонент            | Содержание                                                                   | Данные / API                   |
|----------------------|------------------------------------------------------------------------------|--------------------------------|
| KPI-карточки (4 шт.) | Выручка LTM · Активных клиентов · Средний чек · Churn Risk Rate              | GET /api/dashboard/kpi         |
| График выручки       | Recharts LineChart: факт + прогноз AI, 24 мес.; аномалии подсвечены маркером | GET /api/dashboard/revenue     |
| Список действий AI   | Приоритизированный список: кому написать сегодня, причина, цель, churn score | GET /api/clients/action-list   |
| AI Insights          | Автогенерируемые тезисы от LLM; обновление 1 раз в сутки (Celery крон)       | GET /api/insights (кэш 24 ч)   |
| RFM-матрица          | Bubble chart: X=Recency, Y=Frequency, размер пузыря=Monetary; клик → фильтр  | GET /api/clients/rfm           |
| Сезонность           | Bar chart по месяцам: реальные отклонения (июнь −9.7%, декабрь +7%)          | GET /api/dashboard/seasonality |

Список действий AI — центральный элемент дашборда. Каждое утро Celery-задача запрашивает Churn Engine и LLM, формирует JSON-список (client\_id, reason, goal, churn\_score, priority), коммитит в CoreGit и кэширует в Redis на 24 часа. Фронт читает из кэша.

### 5.3 Карточка клиента

Клик на строку в любой таблице или списке действий открывает боковую панель (Sheet) с карточкой клиента:

- Основные реквизиты: название, ИНН, сегмент RFM, регион
- Метрики: LTV, средний чек, частота заказов, последний заказ
- Churn Score: шкала 0–100% с цветовой индикацией
- История заказов: таблица за последние 12 месяцев

- AI-комментарий: одним абзацем — что нужно сделать с этим клиентом и почему
- Быстрые действия: кнопка «Открыть чат» → переход в AI-чат с предзаполненным клиентом

5.4 API-эндпоинты дашборда

| Эндпоинт                       | Метод | Описание                                         |
|--------------------------------|-------|--------------------------------------------------|
| GET /api/dashboard/kpi         | GET   | 4 метрики + дельты к прошлому периоду            |
| GET /api/dashboard/revenue     | GET   | Временной ряд выручки, 24 мес., + прогноз        |
| GET /api/dashboard/seasonality | GET   | Сезонные индексы по месяцам                      |
| GET /api/clients/action-list   | GET   | Список "кому написать" (кэш Redis 24 ч)          |
| GET /api/clients/rfm           | GET   | RFM-данные: coordinates + segment + monetary     |
| GET /api/clients/:id           | GET   | Карточка клиента: полные данные + AI-комментарий |
| GET /api/insights              | GET   | AI-тезисы (ежедневная генерация, Celery)         |
| POST /api/export/report        | POST  | Запуск генерации PDF-отчёта (async)              |

Все GET-эндпоинты кэшируются в Redis с TTL: KPI и revenue — 1 час, action-list и insights — 24 часа. При пустом кэше — fallback на синхронный запрос к PostgreSQL. Это защищает дашборд от деградации при медленном LLM-ответе.



## ИНТЕРФЕЙС 2: AI-ЧАТ С ПРЕСЕТАМИ

### 6.1 Layout и шаблон

За основу берётся chat-layout от shadcn/ui (или генерируется через v0.dev): левая боковая колонка с историей сессий, основная область чата, фиксированный нижний блок с полем ввода и горизонтальной лентой пресетов.

| Область                   | Содержание                                                                  |
|---------------------------|-----------------------------------------------------------------------------|
| Sidebar (левая панель)    | История чатов пользователя по дате; клик → загружает контекст сессии        |
| Main (центр)              | Поочерёдные сообщения пользователя и AI; стриминг ответа через SSE          |
| Bottom bar (нижняя часть) | Поле ввода + кнопка отправки + горизонтальная прокручиваемая лента пресетов |

Пресеты оформлены как chip-кнопки с иконкой и кратким названием. При клике — текст вставляется в поле ввода (режим: вставить для редактирования) либо отправляется сразу (настраивается в профиле пользователя). Категории разделены визуальными разделителями.

### 6.2 Пресеты промтов (28 шаблонов)

Пресеты сгруппированы по 5 категориям и хранятся в CoreGit-репозитории (presets.json). Администратор может обновить пресеты через CoreGit SDK без перезапуска приложения.

#### Категория 1: Анализ клиента (6 пресетов)

| № | Название пресета   | Текст шаблона                                                                                                                 |
|---|--------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 1 | Полный профиль     | Покажи полный профиль клиента [название/ИНН]: история заказов, средний чек, сегмент RFM, churn score и рекомендацию по работе |
| 2 | Последний заказ    | Когда клиент [название] последний раз делал заказ, на какую сумму и что заказывал?                                            |
| 3 | Churn-анализ       | Каков риск оттока клиента [название]? Объясни, что стоит за этим показателем, и что нужно сделать                             |
| 4 | Сравнение клиентов | Сравни клиентов [А] и [Б] по ключевым метрикам: выручка, частота, средний чек, churn score                                    |
| 5 | Топ продуктов      | Какие продукты и категории чаще всего заказывает клиент [название]? Есть ли изменения за последние 6 месяцев?                 |
| 6 | Сезонность клиента | Есть ли у клиента [название] сезонность в покупках? В какие месяцы он обычно активен или неактивен?                           |

#### Категория 2: Подготовка к контакту (5 пресетов)

| № | Название пресета        | Текст шаблона                                                                                                             |
|---|-------------------------|---------------------------------------------------------------------------------------------------------------------------|
| 7 | Скрипт реактивации      | Подготовь скрипт звонка клиенту [название]: цель — реактивация после длительного молчания. Учти историю заказов и сегмент |
| 8 | Падение чека            | Что сказать клиенту [название], у которого средний чек упал на [%]? Предложи аргументы и вопросы для диалога              |
| 9 | Удержание от конкурента | Подготовь аргументы для удержания клиента [название] от                                                                   |

| №  | Название пресета      | Текст шаблона                                                                                          |
|----|-----------------------|--------------------------------------------------------------------------------------------------------|
|    |                       | перехода к конкуренту. Что у нас сильнее?                                                              |
| 10 | План встречи          | Составь план встречи с клиентом [название] для обсуждения расширения поставок. Ключевые темы и вопросы |
| 11 | Работа с возражениями | Какие возражения может выдвинуть клиент [название] и как на них ответить? Учти его профиль и историю   |

### Категория 3: Аналитика и тренды (5 пресетов)

| №  | Название пресета     | Текст шаблона                                                                                                |
|----|----------------------|--------------------------------------------------------------------------------------------------------------|
| 12 | Рост заказов         | Какие клиенты показали рост заказов за последние 3 месяца? Топ-10 по динамике                                |
| 13 | Топ по выручке       | Топ-20 клиентов по выручке за последний квартал с разбивкой по сегментам RFM                                 |
| 14 | Высокий churn в Core | Покажи клиентов с churn score выше 70% среди Core-сегмента. Что их объединяет?                               |
| 15 | Динамика RFM         | Как изменилась структура RFM-сегментов за последние 6 месяцев? Какие клиенты переместились между сегментами? |
| 16 | Прогноз выручки      | Каков прогноз выручки на следующий месяц? Объясни ключевые факторы и возможные отклонения                    |

### Категория 4: Сегментация и таргетинг (6 пресетов)

| №  | Название пресета    | Текст шаблона                                                                                                       |
|----|---------------------|---------------------------------------------------------------------------------------------------------------------|
| 17 | Апселл в сегменте В | Какие клиенты из сегмента В готовы к апселлу? Критерии: рост частоты заказов, низкий churn, незаполненный потенциал |
| 18 | Перевод В → А       | Покажи список клиентов, которых можно перевести из сегмента В в А по текущей динамике покупок                       |
| 19 | Оптовые условия     | Кому из региональных клиентов стоит предложить оптовые условия? Анализ объёмов и частоты                            |
| 20 | Спящий потенциал    | Найди клиентов с высоким потенциалом (объём в прошлом), которых мы не контактировали более 60 дней                  |
| 21 | Новые позиции       | Кому из клиентов стоит предложить новую позицию [название продукта]? На основе истории и сегмента                   |
| 22 | Кластер риска       | Сформируй кластер клиентов с одновременным высоким churn score и снижением среднего чека — приоритет на спасение    |

### Категория 5: Планирование и коммуникации (6 пресетов)

| №  | Название пресета      | Текст шаблона                                                                                                          |
|----|-----------------------|------------------------------------------------------------------------------------------------------------------------|
| 23 | Список на неделю      | Составь список клиентов для звонков на эту неделю с приоритетами. Учти churn score, дату последнего контакта и сегмент |
| 24 | Антисезонная кампания | Какую кампанию стоит запустить в [месяц] с учётом сезонного спада? Предложи целевых клиентов и формат                  |
| 25 | WhatsApp-реактивация  | Напиши WhatsApp-сообщение клиенту [название] —                                                                         |

| №      | Название пресета         | Текст шаблона                                                                                                             |
|--------|--------------------------|---------------------------------------------------------------------------------------------------------------------------|
| 5      |                          | напомни о себе после долгого молчания. Тон: дружелюбный, не навязчивый                                                    |
| 2<br>6 | Коммерческое предложение | Составь короткое коммерческое предложение для клиента [название] с учётом его истории заказов и сегмента                  |
| 2<br>7 | Рассылка для сегмента    | Напиши письмо-реактивацию для клиентов, не делавших заказ 90+ дней. Сегмент: [В/С]. Тон: [формальный/дружелюбный]         |
| 2<br>8 | Месячный план            | Помоги составить план работы с клиентской базой на следующий месяц: приоритеты, кому звонить, кому писать, что предложить |

6.3 Технические особенности чата

| Аспект             | Реализация                                                                                               |
|--------------------|----------------------------------------------------------------------------------------------------------|
| Стриминг ответов   | Server-Sent Events (SSE) — ответ LLM появляется токен за токеном, без ожидания полного ответа            |
| Контекст разговора | Каждая сессия хранит историю (session_id); LLM получает последние N сообщений в системном промте         |
| RAG-обогащение     | При упоминании клиента в запросе — AI Core автоматически подтягивает его данные из PostgreSQL в контекст |
| История чатов      | Сессии хранятся в PostgreSQL; sidebar отображает по дате; клик загружает контекст                        |
| Пресеты из CoreGit | Frontend загружает presets.json через API Gateway; обновления без перезапуска                            |
| Режим пресета      | Настраивается в профиле: "вставить в поле" (для редактирования) или "отправить сразу"                    |

SSE-стриминг критичен для UX: без него пользователь ждёт 5–15 секунд перед появлением ответа. С SSE первые токены появляются через 0.5–1 сек. Реализация: FastAPI StreamingResponse + asyncio generator → React EventSource.

6.4 API-эндпоинты чата

| Эндпоинт                      | Метод  | Описание                                                      |
|-------------------------------|--------|---------------------------------------------------------------|
| POST /api/chat/message        | POST   | Отправить сообщение; возвращает SSE-стрим (text/event-stream) |
| GET /api/chat/sessions        | GET    | История сессий текущего пользователя                          |
| GET /api/chat/sessions/:id    | GET    | Сообщения конкретной сессии                                   |
| DELETE /api/chat/sessions/:id | DELETE | Удалить сессию из истории                                     |
| GET /api/chat/presets         | GET    | Список пресетов (из CoreGit / кэш)                            |

BACKEND-СЛОЙ: API GATEWAY И СЕРВИСЫ

7.1 Архитектура сервисов

Три сервиса с чёткими зонами ответственности. Принцип из Allosta v2.0: не плодить сервисы — добавлять только при реальной необходимости.

| Сервис       | Технология        | Зона ответственности                                      | Порт / адрес    |
|--------------|-------------------|-----------------------------------------------------------|-----------------|
| Nginx        | Nginx 1.25        | Реверс-прокси: /app → Frontend, /api → Gateway            | 443 (TLS)       |
| API Gateway  | FastAPI + Uvicorn | JWT-аутентификация, маршрутизация, rate limiting          | 8000 (internal) |
| Async Worker | Celery + Redis    | Фоновые задачи: AI Insights, экспорт PDF, CoreGit-коммиты | celery worker   |
| Redis        | Redis 7           | Кэш (KPI, insights) + брокер Celery                       | 6379 (internal) |

AI Core (LLM+RAG) и PostgreSQL — существующие сервисы, не изменяются. API Gateway обращается к ним через внутренние HTTP-вызовы. Churn Engine также вызывается как HTTP-сервис (паттерн Allosta: слабая связанность, независимое масштабирование).

7.2 Аутентификация и роли

JWT-токены, выпускаемые API Gateway при логине. Два профиля доступа:

| Роль    | Доступные интерфейсы                           | Ограничения                                                         |
|---------|------------------------------------------------|---------------------------------------------------------------------|
| analyst | Дашборд + AI-чат (полный)                      | Нет доступа к настройкам порогов и управлению пользователями        |
| sales   | Только AI-чат                                  | Не видит дашборд и отчёты; доступ только к пресетам и своей истории |
| admin   | Дашборд + чат + настройки + CoreGit-управление | Полный доступ; управление пороговыми, пресетами, пользователями     |

7.3 Celery-задачи

| Задача                  | Расписание                       | Действие                                                       |
|-------------------------|----------------------------------|----------------------------------------------------------------|
| generate_daily_insights | Ежедневно в 06:00                | Запрос LLM → формирование тезисов → Redis кэш → CoreGit-коммит |
| generate_action_list    | Ежедневно в 06:30                | Churn Engine → LLM-ранжирование → Redis кэш → CoreGit-коммит   |
| refresh_rfm_scores      | Еженедельно в понедельник        | Пересчёт RFM из PostgreSQL → обновление кэша                   |
| export_pdf_report       | По требованию (POST /api/export) | Puppeteer-рендеринг дашборда → PDF → CoreGit-коммит → ссылка   |
| commit_presets          | При изменении через admin        | Обновление presets.json в CoreGit с коммит-сообщением          |

*Все Celery-задачи идемпотентны: повторный запуск при сбое не создаёт дублей. CoreGit-коммит делается последним шагом; при неудаче задачи sha не записывается в PostgreSQL — система сохраняет согласованность.*

СПРИНТ-ПЛАН (6 НЕДЕЛЬ)

Спринт 1 — Недели 1–2: Фундамент

| Задача                    | Детали                                                                                           |
|---------------------------|--------------------------------------------------------------------------------------------------|
| Форк shadcn/ui шаблона    | Dashboard template (ui.shadcn.com/examples/dashboard) + chat layout; удаление лишних компонентов |
| Инфраструктура            | Docker Compose: Nginx, FastAPI, Redis, Celery; CoreGit-репозиторий создан, API-ключ получен      |
| JWT-аутентификация        | Login-страница, три роли, refresh-токен, защищённые роуты                                        |
| API Gateway: скелет       | Все эндпоинты из разделов 5.4 и 6.4 — заглушки с mock-данными                                    |
| Подключение к PostgreSQL  | Read-only connection pool; базовые запросы KPI (выручка, клиенты)                                |
| CoreGit SDK инициализация | @coregit/sdk подключён в Celery-воркере; тестовый коммит прошёл                                  |

Критерий готовности: приложение открывается в браузере, можно залогиниться с тремя ролями, все эндпоинты отвечают (с mock-данными), CoreGit принимает тестовый коммит.

Спринт 2 — Недели 3–4: Дашборд на реальных данных

| Задача                    | Детали                                                                              |
|---------------------------|-------------------------------------------------------------------------------------|
| KPI-карточки и график     | Реальные данные из PostgreSQL; дельты к прошлому периоду; Recharts LineChart        |
| Список действий AI        | Celery-задача generate_action_list; Redis-кэш; отображение в таблице с приоритетами |
| RFM-матрица               | Bubble chart на реальных сегментах ХБК Аксай; фильтрация таблицы клиентов по клику  |
| Карточка клиента          | Side panel: реквизиты, метрики, churn score, история, AI-комментарий                |
| AI Insights               | Celery-задача generate_daily_insights; вывод тезисов на дашборде                    |
| CoreGit: фиксация выводов | Каждый запуск AI Insights и action_list коммитится в CoreGit с датой                |

Критерий готовности: дашборд показывает реальные данные ХБК Аксай; список действий обновляется автоматически; CoreGit содержит историю AI-выводов.

Спринт 3 — Недели 5–6: AI-чат, пресеты, полировка

| Задача         | Детали                                                                                |
|----------------|---------------------------------------------------------------------------------------|
| Chat layout    | Sidebar с историей, область сообщений, поле ввода; sidebar показывает реальные сессии |
| SSE-стриминг   | FastAPI StreamingResponse → React EventSource; ответ AI появляется токен за токеном   |
| RAG-обогащение | При упоминании клиента в запросе — автоматическое добавление его данных в контекст    |
| 28 пресетов    | Chip-компоненты в нижней ленте; загрузка из CoreGit (presets.json); 5 категорий       |

| Задача         | Детали                                                                                 |
|----------------|----------------------------------------------------------------------------------------|
| Экспорт отчёта | POST /api/export → Celery-задача → Puppeteer PDF → CoreGit-коммит → ссылка             |
| QA и пилот     | Тестирование с командой продаж ХБК Аксай; фиксация обратной связи; финальная полировка |

*Критерий готовности: оба интерфейса работают на реальных данных; SSE-стриминг стабилен; 28 пресетов протестированы командой продаж; CoreGit содержит полную историю выводов.*

## КРИТИЧЕСКИЕ РЕШЕНИЯ И РИСКИ

| Решение / Риск                       | Статус                         | Рекомендация                                                                                                                                                        |
|--------------------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LLM-хост: локальный сервер vs облако | Требуется решения до Спринта 1 | Локальный (Llama3/Mistral) — данные не уходят, но выше задержка стриминга (2–5 сек vs 0.5 сек). Если задержка неприемлема — OpenAI/Anthropic с шифрованием запросов |
| Один домен vs два поддомена          | Требуется решения до Спринта 1 | Рекомендуется один домен: /dashboard и /chat — проще SSL, нет CORS-проблем                                                                                          |
| Хранение истории чатов               | Решено                         | PostgreSQL (уже есть); отдельная Redis-очередь избыточна                                                                                                            |
| Puppeteer для PDF                    | Потенциальный риск             | Puppeteer требует headless Chrome в контейнере; альтернатива — WeasyPrint (Python, легче)                                                                           |
| CoreGit API-ключ                     | Требуется действия             | Зарегистрировать на coregit.dev, получить cgk_live_*** ключ, добавить в env                                                                                         |
| Мобильная адаптация                  | Частично                       | shadcn/ui responsive из коробки; дашборд — desktop-first, чат — адаптирован под планшет                                                                             |
| shadcn/ui лицензия                   | Решено                         | MIT License; нет ограничений на коммерческое использование                                                                                                          |

Рекомендуется начинать Спринт 1 немедленно после фиксации решения по LLM-хосту и домену. Остальные решения не блокируют старт и могут быть приняты по ходу работы.

### 9.1 Вне рамок Этапа 2 (Этап 3)

Следующие компоненты намеренно исключены из Этапа 2 во избежание scope creep:

- Push-уведомления через Telegram-бот для менеджеров (добавляется в Этапе 3 за 1–2 дня при наличии Bot API)
- PWA / мобильное приложение (React Native) — требует отдельного sprint'a
- Двусторонняя синхронизация с 1С — Этап 2 работает только в read-only режиме
- Голосовой интерфейс — не в приоритете до накопления статистики использования чата



**ЗАКЛЮЧЕНИЕ**

Этап 2 AI Sales Director представляет собой логически завершённую инфраструктурную надстройку над работающим AI-ядром. Принципиальные решения — использование готового shadcn/ui-шаблона, единый PostgreSQL без новых зависимостей, паттерны API Gateway и asynс-воркера из архитектуры Allostа v2.0 — обеспечивают предсказуемые 6 недель реализации без архитектурных рисков.

CoreGit встраивается в архитектуру в своей естественной роли: версионированное git-хранилище AI-выводов, не backend-платформа. Это обеспечивает полный аудит всех рекомендаций системы — критически важно для корпоративной эксплуатации.

По завершении Этапа 2 сотрудники ХБК Аксай получают два инструмента: Дашборд аналитика с автоматическим ежедневным списком приоритетных клиентов и AI-чат с 28 проверенными шаблонами запросов. Оба инструмента работают на данных, уже накопленных системой, без ручного ввода.

| Итоговые параметры плана                                           |
|--------------------------------------------------------------------|
| <b>Срок:</b> 6 недель (3 спринта)                                  |
| <b>Интерфейсы:</b> 2 (Дашборд аналитика + AI-чат)                  |
| <b>Пресеты:</b> 28 шаблонов в 5 категориях                         |
| <b>Новых сервисов:</b> 3 (API Gateway + Celery Worker + Redis)     |
| <b>Изменений в PostgreSQL / AI Core:</b> 0                         |
| <b>CoreGit:</b> Версионирование AI-выводов, пресетов, конфигураций |